Counting Ability of Large Language Models and Impact of Tokenization

Xiang Zhang[1*] **Juntai Cao**[1*] **Chenyu You**[2]
[1] University of British Columbia
[2] Stony Brook University
xzhang23@ualberta.ca, jtcao7@cs.ubc.ca, chenyu.you@stonybrook.edu

arXiv:2410.19730v2 [cs.CL] 29 Oct 2024

Abstract

Transformers, the backbone of modern large language models (LLMs), face inherent architectural limitations that impede their reasoning capabilities. Unlike recurrent networks, Transformers lack recurrent connections, confining them to constant-depth computation. This restriction places them in the complexity class TC^0, making them theoretically incapable of solving tasks that demand increasingly deep reasoning as input length grows. Counting, a fundamental component of many reasoning tasks, also requires reasoning depth to grow linearly to be performed inductively. While previous studies have established the upper limits of counting ability in Transformer-based expert models (i.e., models specifically trained for counting tasks), these findings do not directly extend to general-purpose LLMs due to differences in reasoning mechanisms. Recent work has highlighted how Chain of Thought (CoT) reasoning can help alleviate some of the architectural limitations of Transformers in counting tasks. However, little attention has been paid to the role of tokenization in these models. Unlike expert models that often use character-level tokenization, LLMs typically rely on byte-level (BPE) tokenizers, which fundamentally alters the way reasoning is processed. Our work investigates the impact of tokenization on the counting abilities of LLMs, uncovering substantial performance variations based on input tokenization differences. We provide both theoretical and experimental analyses, offering insights into how tokenization choices can undermine models' theoretical computability, thereby inspiring the design of new tokenization methods to enhance reasoning in LLMs. All code, prompts, and experiment logs, API returns are released on GitHub.

1 Introduction

Counting, a fundamental component of most complex reasoning tasks, has been extensively studied across various disciplines for decades (Boolos et al., 2002; Wynn, 1990; De Bruijn, 1964). In particular, computer scientists have explored the circuit complexity of counting (Jerrum, 1995), the needed capabilities of computing machines (Fischer et al., 1968) to perform counting, and how counting relates to more complex tasks within the framework of computability theory (Ibarra et al., 2002). Basic counting (counting from `1` to `n`) requires a depth complexity, the number of sequential computation steps, that grows with input length (non-constant). This requirement, grounded in computability theory (Fischer et al., 1968), serves as a theoretical constraint for any computational machine to solve this task, from simple state machines (Cooper, 2017) to neural networks (LeCun et al., 2015; Jin et al., 2024). Previous research has thoroughly examined the computability, the ability to solve tasks of a given complexity, of different types of neural networks (Delétang et al., 2022; Zhang et al., 2024a). Both theoretical (Raghu et al., 2017) and experimental (Delétang et al., 2022) findings have shown that *counting* requires an equivalent or higher level of computability than what LSTM and RNN provide (Delétang et al., 2022), with the former empirically aligning with the capabilities of `k`-counter machines (Delétang et al., 2022).

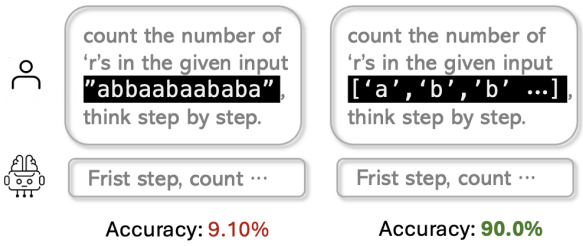

Figure 1: Experimental results on average counting accuracy based on different tokenization choices, using GPT-4o mini. Our approach treats the model as a *black-box*, manipulating BPE tokenizers to function differently through carefully engineered string formats.

*Equal contribution.

Transformers (Vaswani, 2017), both autoregressive (Gregor et al., 2014; Achiam et al., 2023; Zhang et al., 2023) (GPT-like) and non-autoregressive (Devlin, 2018; Liu et al., 2022; Zhang et al., 2024b) (BERT-like), are constrained to the *constant-depth* computation (Zhang et al., 2024a; Delétang et al., 2022; Li et al., 2024). Neural networks perform all computations and reasoning in a latent space, $\mathcal{H}$, where the hidden state $\mathbf{h}$ serves as the medium for computational state storage and information processing. Unlike recurrent architectures, where $\mathbf{h}_{\mathtt{t}} = g_\theta(\mathbf{h}_{\mathtt{t-1}})$ allows hidden states to evolve over time, Transformers update $\mathbf{h}$ sequentially only through layers rather than time steps. Consequently, Transformer with a fixed number of layers can only process (or reason with) $\mathbf{h}$ a constant number of times, placing them at the lower end of the Chomsky computational hierarchy (Delétang et al., 2022). This architectural limitation prevents Transformer-based models—from small expert model to LLMs—from performing *counting* tasks relying solely on their internal hidden states computation.

Chain of Thought (Wei et al., 2022) reasoning has revolutionized the mechanism by which reasoning can be performed. Specifically, it *extends* reasoning from the latent space $\mathcal{H}$ to the text space $\mathcal{O}$ (Zhang et al., 2024a), using sequences of natural language (referred to as "*Thought*") to relay the computations of $\mathbf{h}$ in the absence of recurrence. As a result, higher-complexity tasks, such as counting, become feasible. Several theoretical works (Li et al., 2024; Zhang et al., 2024a; Feng et al., 2024) have proven that CoT-augmented LLMs have an *upper-bound* (under ideal assumptions) for performing computational tasks of arbitrary complexity, including *counting*.

Despite extensive analysis and theoretical guarantees on the upper bound for counting ability, a significant gap remains in practical performance (Zhang et al., 2024a; Chang and Bisk, 2024). As modern LLMs scale from millions to billions of parameters (Achiam et al., 2023), improvements in counting ability have been minimal—GPT-4, for example, struggles with simple tasks like counting the number of "r"s in a word. While recent research has examined factors such as training data selection (Allen-Zhu and Li, 2023; Yin et al., 2023) and positional encoding (Chang and Bisk, 2024), one of the most fundamental aspects, tokenization, has received relatively little attention. Specifically, modern byte pair encoding (BPE) (Sennrich, 2015) tokenization for LLMs groups multiple characters into a single token for efficiency, which will inadvertently degrade performance in arithmetic tasks due to information loss during the tokenization process.

In this work, we thoroughly investigate how the choice of tokenization can significantly undermine the theoretical counting ability of neural models. We adopt a model-agnostic approach, allowing for the analysis of closed-source LLMs with unknown tokenizers. Through extensive counting experiments using CoT (theoretically proven to be Turing complete under ideal conditions), we demonstrate that the correct tokenization choice is crucial for fully out-bringing a model's theoretical counting ability and bridging the gap between theory and practice. Otherwise, accuracy drops of up to 80% can be observed. Additionally, our experiments reveal that the impact of tokenization varies across models, with certain tokens proving more sensitive in counting tasks, even when the nature of the task remains unchanged.

2 Neural Networks and Counting: A Revisit

Training neural networks for counting. Counting has been extensively studied in neural networks (NNs) as it is a fundamental skill required for more advanced tasks (Chang and Bisk, 2024). Since multi-layer perceptrons (MLPs) (Rosenblatt, 1958) can only handle fixed-length inputs, which contradicts the nature of counting, early NN training for counting tasks began with recurrent neural networks (RNNs). Rodriguez et al. (1999) trained an early RNN to recognize the *regular* language $\mathtt{a}^n\mathtt{b}^n$, which required the network to count the number of $\mathtt{a}$s and $\mathtt{b}$s in the input string. Of the 50 networks trained, 8 successfully learned the task and generalized to longer strings, demonstrating the counting ability of RNNs. Suzgun et al. (2019) trained LSTM for counting in the context of *bracket pairing*, showing that LSTMs could perform *dynamic counting* by maintaining separate counters (represented by different types of brackets) using their memory (cell state) and gating mechanisms—something RNNs were unable to achieve. More recently, Delétang et al. (2022) systematically explored counting in mainstream NNs, including RNNs, LSTMs, and Transformers. Their experiments showed that Transformers could not

perform counting, while LSTMs exhibited computability aligned with *counter machines*. Chang and Bisk (2024) extended the analysis of counting performance in Transformers to other modern architectures such as Mamba (Gu and Dao, 2023) and RWKV (Peng et al., 2023), demonstrating that these models also struggle with counting tasks outside their training distribution, performing worse than RNNs empirically.

Analyzing neural networks' ability to count. Counting falls within the domain of computability, which studies a machine's ability to perform tasks of varying complexity. Researchers have made efforts to theoretically understand the upper bounds of each type of neural network from a computability perspective. Sperduti (1997) and Tiňo et al. (1998) were among the first to theoretically align RNNs with deterministic finite automata (DFA), showing that, under proper assumptions, they are equivalent. In this context, RNNs act as one-finite-counter machines, capable of performing basic counting tasks (e.g., parity checks). Recent studies (Weiss et al., 2018; Ackerman and Cybenko, 2020) offer further insights and more rigorous proofs on the computability of RNNs and LSTMs, theoretically confirming earlier experimental results on their counting abilities. However, Transformers, lacking inherent recurrence, are limited to TC^0 (Li et al., 2024) in terms of inductive reasoning. As demonstrated in recent theoretical work (Sanford et al., 2024; Li et al., 2024; Delétang et al., 2022), this places Transformers at the lowest level of the Chomsky hierarchy, making it impossible for them to learn counting tasks unless certain inductive bias is introduced(Chang and Bisk, 2024).

3 CoT + Ideal Assumption = Complete Counting Ability

Mainstream LLMs use the Transformer architecture as their backbone (Bai et al., 2023; Touvron et al., 2023; Achiam et al., 2023), and, as a result, inherit the computability limitations of Transformers, particularly in tasks like counting. Given these constraints, it's unsurprising that Transformers struggle with counting tasks. However, Chain of Thought (CoT) reasoning has revolutionized how Transformers reason, offering new possibilities counting task. In this section, we demonstrate *what* counting entails and *why* Transformers alone cannot effectively solve counting tasks. We then show *how* CoT can achieve perfect counting accuracy under ideal assumptions.

3.1 Inductive Counting and Computation

Counting is inherently inductive in both human cognition and computability theory (Borodin et al., 1989). To count from `1` to `n`, one must start at `1`, then proceed step by step—counting to `2` before reaching `3`, and so on. Humans typically count inductively, except for very small numbers (1-3), where we rely on memorization and can immediately recognize the total at a glance. For larger quantities, like a pile of apples, we engage in iterative inductive counting from 1 to `n` (Figure 2). In computability, a state machine processes an input string token by token, transitioning its internal state after each token to keep track of the count. This step-by-step state transition aligns with the inductive nature of counting. Similarly, neural networks perform counting internally through their hidden state, $\mathbf{h}$, which serves as the location for reasoning and intermediate information storage (e.g., counter storage) (Figure 2. In RNNs, as $\mathbf{h}$ is updated with each new input token x_t via recurrent connections, the counter can be inductively updated at every step (Figure 2). The inductive bias of RNNs allows $\mathbf{h}$ to act as the smallest *reasoning unit*, incrementing the stored counter with each update.

However, Transformers can only sequentially process $\mathbf{h}$ a fixed number of times (Li et al., 2024; Sanford et al., 2024; Zhang et al., 2024a), limited by their number of layers. For instance, when counting the number of `a`s in the input string `aababaa`, the Transformer initializes its counter in the latent representation $\mathbf{h}$. While each layer captures substantial computation through matrix operations ($\mathbf{WX}$), because $\mathbf{h}$ is computed in parallel across positions `t`, it lacks the depth (sequential counting) needed for inductive counting. The counter is only updated when the hidden state $\mathbf{h}$ is sequentially passed from one layer to the next, limiting the model to a fixed number of counting steps. Some theoretical work (Chiang and Cholak, 2022) suggests that Transformers can perform counting with hard-coded weights, but this approach is not *inductive*. Instead, it treats each bit in $\mathbf{h}$ and $\mathbf{W}$ as the smallest unit of reasoning (in contrast to treating the entire $\mathbf{h}$ as a unit in RNNs) and performs *circuit-level* computations. This relies on the bit-level dependencies within matrix multiplication (e.g., $w_1x_1 + w_2x_2 + \cdots + w_nx_n$),

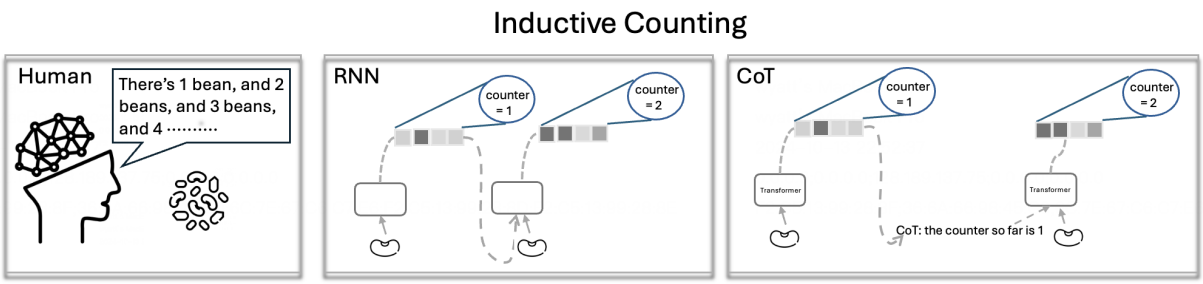

Figure 2: Illustration of inductive counting as performed by humans, RNNs, and LLMs with CoT, respectively.

breaking the abstraction of neural computation and delving into circuit-level operations. As a result, training a standard Transformer, as demonstrated by various experimental studies (Chang and Bisk, 2024; Delétang et al., 2022), consistently fails to learn counting in a generalizable manner, unlike RNNs. Transformers do not converge on these "theoretically possible" counting solutions unless additional inductive biases are introduced, such as specific positional encodings or architectural modifications (Chang and Bisk, 2024; Fan et al., 2020).

3.2 CoT: Sky is the Limit

As Transformer-based models, both LLMs and expert models, struggle to count effectively using only their internal reasoning state (Chang and Bisk, 2024; Delétang et al., 2022) (also demonstrated in experiments below), Chain of Thought (CoT) (Wei et al., 2022) shifts the reasoning required for inductive counting into the text space (Li et al., 2024; Zhang et al., 2024a). Instead of simply outputting a final counting result $\mathtt{y}$ after processing an input sequence $\mathbf{x} = (\mathtt{x}_1, \mathtt{x}_2, \mathtt{x}_3, \cdots)$, LLMs are guided to output intermediate reasoning steps, in this case counter value, after processing each input unit $\mathtt{x}_i$. Since internal reasoning via $\mathbf{h}$ can only handle a limited number of sequential counting steps, CoT allows LLMs to convert the latent counter information from $\mathbf{h}$ into a sequence of tokens $(\mathtt{o}_1, \mathtt{o}_2, \cdots, \mathtt{o}_k)$, referred to as *Thought*, which represents the counter value in text (Figure 2). During subsequent computations, these *thought* tokens are encoded back into the latent space through the embedding layer, whose information forming the new $\mathbf{h}$ for the next step of reasoning (Figure 2). In essence, CoT approximates the recurrent computation in RNNs—$\mathbf{h}_{t-1} \Rightarrow \mathbf{h}_t$—by using $\mathbf{h}_{t-1} \Rightarrow (\mathtt{o}_1, \mathtt{o}_2, \cdots, \mathtt{o}_k) \Rightarrow \mathbf{h}_t$, where $(\mathtt{o}_1, \mathtt{o}_2, \cdots, \mathtt{o}_k)$ encodes information from $\mathbf{h}_{t-1}$. This enables infinite reasoning depth when the CoT can be extended indefinitely (ideal assumption). Consequently, CoT allows counting to be performed iteratively and inductively, using constant cycle of text-vector conversion to continuously update the counter (Zhang et al., 2024a).

4 Tokenization as a Black Box Model

Even with CoT enabled, significant failures in counting are still observed in modern LLMs like GPT-4, which consistently make errors when counting letters in words as short as 3 to 10 characters such as `Strawberry` (Table 1). Given the number of layers in models of this size (Touvron et al., 2023), counting within 10 digits internally should be feasible. A key factor contributing to these errors is the tokenizer used, specifically byte-level Byte Pair Encoding (BPE) (Sennrich, 2015). BPE groups a certain number of characters into tokens, both within and between words, leading to a mismatch between the unit to be counted (e.g., letters) and the unit actually being processed (BPE tokens). In this section, we explore the potential impact of tokenization on the counting abilities of LLMs and introduce our novel method for analyzing this effect, treating the model as a *black box*.

4.1 Imperfect Tokenizer + CoT < CoT Limit

We introduce the concept of *Token Awareness* in the context of LLMs. For any given token $\mathbf{t}$ (e.g., the token `pre`) produced after tokenization, each token has properties such as `how many "r"s are in this token`. Since LLMs are trained to predict entire *tokens* rather than individual *letters*, specific token properties may not be fully *aware* by the model unless they can be inferred from the training corpus. During training, a token is mapped directly into its token embedding without additional information, meaning the model may not even know if this token contain any letters.

Now, consider a CoT-augmented LLM under ideal conditions, where it has perfect counting ability. If the counting is performed at a level that does

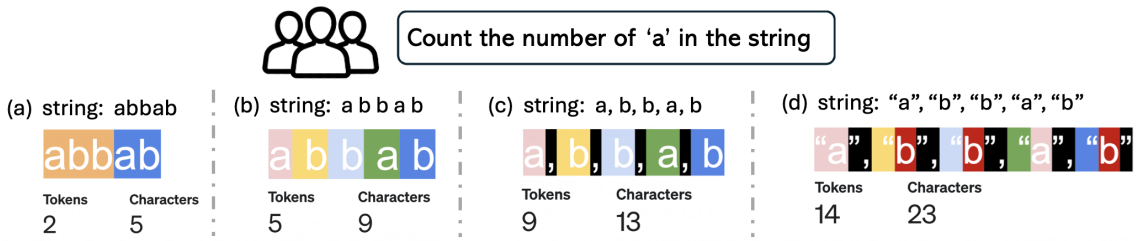

Figure 3: Four types of string formatting used for counting tasks to manipulate tokenization in LLMs. Examples in the figure are tokenized using the GPT-4o tokenizer. Each string-token type is labeled as (a), (b), (c), and (d) in the diagram. Note that changing the format does not alter the fundamental nature or difficulty of the counting task.

not align with the tokenization—such as counting letters instead of BPE tokens—missing token awareness for the required granularity will lead to counting errors. For instance, if the `i`th token in the input string has token properties that the model is unaware of, counting at token `i` will cause a shift in the total count. As an example, if the model is unaware of the number of `rs` in the first token `str` when processing the word `strawberry`, the counting will fail right from the start. Thus, even with perfect *Counting Ability*, tokenization becomes a barrier to successful task performance.

4.2 Method

Building on the theoretical analysis, our goal is to examine the practical counting abilities of LLMs, with a particular focus on the impact of tokenization—a crucial yet often under-explored factor in a model's counting potential. We propose a novel approach to studying the role of tokenization in LLMs' counting performance. Since many LLMs, along with their tokenization algorithms, are closed-source, we designed our approach to be model-agnostic, treating LLMs as black-box systems.

We begin by establishing universal assumptions based on typical LLM design in real-world models: (1) Consecutive letters of length 2-4 often form a single token; (2) Inputs beside delimiters like commas or spaces, are split into separate tokens. (3) After tokenization, delimiters (e.g., spaces, commas) are often merged with the preceding or following token. However, adding consecutive delimiters can prevent this merging and separate delimiter tokens from adjacent text tokens.

We validated our assumptions using multiple modern open-source LLMs or models with open tokenizers (Figure 3 as well as Appendix Figure 7). Based on these assumptions, we designed input instances for counting tasks to manipulate how tokenization is applied. To highlight the impact of tokenization on counting performance, we focused on letter-level counting, where the model needs token-level letter awareness to correctly update the counter in $\mathbf{h}$. It is important to note that the granularity of counting (letter vs. word level) does not change the fundamental nature of the counting task and should not affect the theoretical limits of counting ability.

To test how naive BPE tokenization affects counting, we designed counting instances as strings of consecutive letters, which are naturally tokenized by merging every 2-4 letters into a single token, based on assumption (1). Figure 3.(a) illustrates the tokenization of such an example using the GPT-4o tokenizer. Formally, for a string $\mathbf{s}$ consisting of $\mathbf{n}$ letters, $(\mathrm{l}_1, \mathrm{l}_2, \cdots, \mathrm{l}_\mathrm{n})$, the task is to count the total occurrences of a target letter $\mathrm{l}_{\mathtt{target}}$:

$$\mathbf{a} \in \mathbb{N}, \quad \mathbf{a} = \sum_{i=1}^{\mathrm{n}} \mathbb{1}_{\{\mathrm{l}_\mathrm{i} = \mathrm{l}_{\mathtt{target}}\}} \tag{1}$$

where $\mathbf{a}$ is the answer (count), and $\mathbb{1}$ is the indicator function, which returns `1` if the current letter matches the target letter.

As this approach inevitably merges consecutive letters, we introduce two alternative methods of inserting between-letter delimiters to simulate cases where `item-separated tokenization` is applied to the same counting instance. We use two types of delimiters, d_1 = `" "` and d_2 = `", "`, as shown in Figure 3.(b)-(c). According to assumptions (2) and (3), the resulting string $(\mathrm{l}_1, \mathrm{d}, \mathrm{l}_2, \mathrm{d}, \cdots, \mathrm{d}, \mathrm{l}_\mathrm{n})$ manipulates the tokenizer to separate each item to be counted into its own token. However, the delimiter d and the letter li will often be merged into a new token $\mathbf{t}$. Accurately counting the instance thus requires token awareness of the merged token with the delimiter.

Lastly, by adding the delimiter "" (quotations) along with ", ", each letter will be separated into its own individual token,

String-Token Type	Counting letter a						Counting letter b					
	len ∈ [10-20]		len ∈ [20-30]		len ∈ [30-40]		len ∈ [10-20]		len ∈ [20-30]		len ∈ [30-40]	
	no-CoT	CoT	no-CoT	CoT	no-CoT	CoT	no-CoT	CoT	no-CoT	CoT	no-CoT	CoT
pure string BPE tokens (a)	30.10	45.70	15.10	9.10	6.40	2.00	33.20	47.70	14.00	9.40	3.80	2.70
" "-deliminated token (b)	46.20	58.40	16.10	24.90	7.50	10.90	45.90	63.70	17.60	34.00	5.60	18.60
", "-deliminated token (c)	56.00	55.40	19.40	38.60	10.20	28.10	63.60	69.30	32.80	56.10	13.90	42.30
precise-item token (d)	50.70	96.80	15.80	81.60	7.90	56.10	58.30	96.50	30.20	90.00	12.60	70.80

Table 1: Resulst of counting as and bs in string consisting of letter a and b, using GPT-4o-mini API. Numbers indicate the average accuracy (%) over 1000 random generated instances.

String-Token Type	Counting letter e						Counting letter z					
	len ∈ [10-20]		len ∈ [20-30]		len ∈ [30-40]		len ∈ [10-20]		len ∈ [20-30]		len ∈ [30-40]	
	no-CoT	CoT	no-CoT	CoT	no-CoT	CoT	no-CoT	CoT	no-CoT	CoT	no-CoT	CoT
pure string BPE tokens (a)	26.60	55.20	19.80	12.20	11.40	2.10	31.10	59.10	11.70	22.10	4.60	7.30
" "-deliminated token (b)	41.00	52.90	23.90	28.20	13.00	16.00	45.30	63.90	16.60	46.20	6.80	29.50
", "-deliminated token (c)	45.50	64.20	27.40	44.20	18.00	27.60	56.20	73.60	28.20	55.60	13.90	41.90
precise-item token (d)	60.10	97.70	32.50	89.30	15.30	70.70	60.60	98.40	30.60	93.80	13.30	74.80

Table 2: Resulst of counting es and zs in string consisting of letter e and z, using GPT-4o-mini model.

string-token	len ∈ [10-20]		len ∈ [20-30]		len ∈ [30-40]	
type	count a	count b	count a	count b	count a	count b
(a)	86.30	86.20	62.40	65.20	50.60	54.40
(b)	90.60	94.00	80.40	87.50	76.10	79.60
(c)	94.90	97.70	92.80	97.90	91.40	94.20
(d)	93.00	94.20	87.80	91.00	87.30	89.80

Table 3: Counting results on strings with letter a and b, using Claude-3.5-sonnet API. All results are using supervised CoT (Zhang and Ding, 2024), with same prompt for GPT-4o-mini.

along with the delimiters. This simulates a case where `precise item-separated tokenization` is applied to the same counting instance.

With this string formatting design, rather than directly changing the tokenizer or applying different tokenizers to the same input, we enable the use of closed-source models, where the "tokenizer" is either unknown or cannot be directly accessed via API by users. Despite this, the desired tokenization effect can still be achieved, as the formatted strings manipulate BPE to behave as intended according to our verified assumptions.

5 Experiments

5.1 Setting

We adopt two mainstream foundation models for our analysis of tokenization and counting ability: GPT-4o mini [1] API and Claude-3.5-sonnet[2] API, considering both affordability and their representation of state-of-the-art performance. To prevent any potential cheating, we purposely disabled the use of *tools* in these LLMs. We used a Python script to generate instances for each of the four types of counting strings demonstrated in Figure 3. Specifically, we generated 1000 strings under three different total length ranges: [10, 20], [20, 30], and [30, 40]. The target length was uniformly sampled within each range, and a random number of letters as and bs were generated to form the complete string for the pure string type (Figure 3.(a)). The other modified string types (Figure 3.(b)-(d)) were derived by adding corresponding delimiters to the pure strings. Labels (correct counts) were generated using Python for both the number of as and the number of bs.

To eliminate potential confounding factors, such as failure to generate correct CoT steps, we use a "supervised CoT" (Zhang and Ding, 2024) approach, where the models are explicitly instructed on what to reason at each step, i.e., the counter value. To ensure fairness, all experiments use the exact same prompt across models and conditions, as detailed in Appendix Section D. The rationale behind the use of Supervised CoT, its comparison with Vanilla CoT in counting tasks, and further details are provided in Appendix Section A.

Lastly, we evaluate the accuracy of the returned counts from the LLMs and report the average counting accuracy for each *tokenization* case. The results are presented in Table 1.

[1] https://chatgpt.com
[2] https://claude.ai

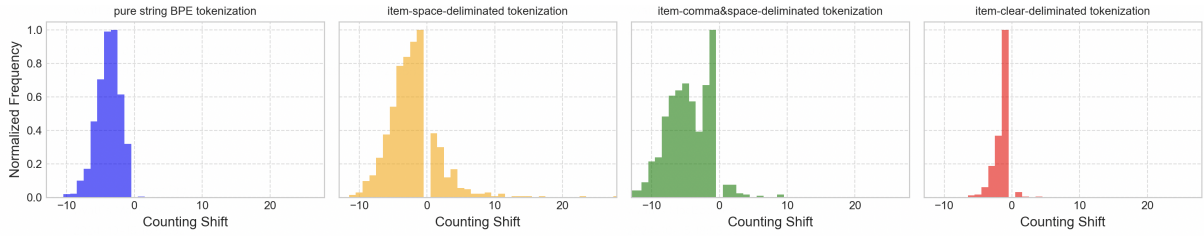

Figure 4: Distribution of shift from correct count to GPT-calculated count, for each type of string-token fomrat (a), (b), (c) and (d) in order. The statsiticas show the results for letter `a` at length range [30, 40], as this range the error rate is high. We only calculate the shift when error is made, as correct counting instance does not have any shift.

5.2 Main Results

We analyze the experimental results alongside our theoretical analysis of the model's upper computability, the role of CoT, and the impact of tokenization, as discussed in the previous section.

5.2.1 CoT Grants Counting Ability

By comparing the results for `no-CoT` and `CoT` across different length ranges and string types, we observe that CoT significantly enhances the model's counting ability in all cases, with an average performance improvement of 20% compared to `no-CoT`.

As discussed in section 2, LMs without CoT, which rely solely on Transformer's internal latent reasoning, can only count a *constant* number of times inductively. This is evident when performance drops sharply as the input length increases from [10, 20] to [30, 40] when CoT is not used. Without CoT, the counting accuracy for letters `a` and `b` falls from around 50% to just 8%, regardless of tokenization, which is barely above random guessing (3-4%). However, with CoT augmentation, counting accuracy remains more stable and declines much more gradually (from 96% drops all the way to 56%), especially with the best tokenization choice (choice (d)). This highlights the crucial role of CoT in unlocking a model's counting potential and the practical gap from theoretical limits, caused by factors such as training quality, long-context retrieval, and CoT length constraints.

5.2.2 Tokenization Greatly Affects Counting Ability

From Table 1, we observe a clear and consistent trend of improved performance as tokenization choices progress from (a) to (d), indicated by increasing color darkness, when the length range is fixed. Notably, **without any** special tokenization (type (a)), counting performance is even worse than not using CoT in the same setting. For string lengths greater than 20, BPE-tokenized strings perform much closer to *random guessing* than to CoT-less models. This highlights how letter-grouped BPE tokenization can *severely* degrade theoretical counting ability due to the lack of token awareness. When item-separated tokenization is applied (using delimiters such as " " or ", " in (b) and (c)), we see consistent improvements (13%-40%) over pure BPE, emphasizing the importance of per-item tokenization rather than grouped tokenization in counting tasks. However, since (b) and (c) still result in items being grouped with delimiters (e.g., spaces), even though the items are separated from one another, the performance limit is only reached when each item (letter) is clearly tokenized, separated from both the delimiters and other items. This eliminates the need for token-awareness, as each item-token (letter) is distinct and ready for comparison with the target, allowing the attention mechanism to function optimally (with higher cosine similarity between identical token embeddings in attention). We conducted the same experiments with a different pair of letters, `e` and `z`, and observed identical patterns. This further demonstrates the significant impact of tokenization on counting ability, with clearly separated targets leading to markedly better results.

5.3 Error Shifts Reveal Mistakes in Counting with BPE Tokenization

We analyze the error shifts, defined as the difference between the model-calculated count and the true count, when mistakes occur. As shown in Figure 4, all tokenization methods result in a bias toward negative shifts, meaning the model underestimates the count. Specifically, when using pure BPE tokenization, we observe *only* negative shifts. This is because BPE-tokenized tokens often contain letters the model is unaware of. For instance, the model might fail to count any "a"s in a single token like "abaa," leading to an undercount. When

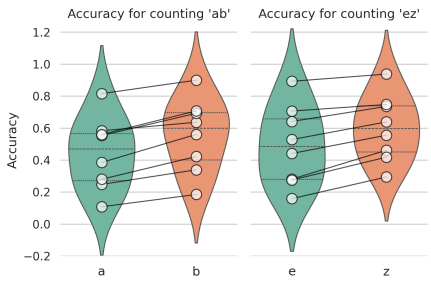

Figure 5: Pairwise comparison of counting accuracy for different letters in strings. The left plot shows the distribution of accuracy for a and b in ab strings, with each dot representing the average accuracy for a in a given CoT case (e.g., spaced-string in the [10,20] range), connected to the corresponding accuracy for b in the same setting. The right plot illustrates a similar case for e and z in ez strings. Note: The y-axis limit exceeds [0,1] as the distribution is calculated based on variance and mean, with larger variance pushing the upper bound of the confidence interval beyond the maximum value.

item-separated tokens are introduced (as in cases (b)-(d)), positive errors begin to appear, caused by factors such as counting mistakes or long-context retrieval errors. However, with clearer token separation (case (d)), extreme shifts disappear, and the errors are mostly centered between -1 and -3, likely due to minor arithmetic mistakes by the model. This shows that while BPE incurs large extreme errors, clearer tokenization greatly mitigates counting errors.

5.4 Different Tokens Have Varying Sensitivity in Counting Tasks

As seen in Table 1, when counting is performed properly (with CoT enabled and tokenization not relying on pure BPE), we consistently observe higher accuracy when counting the letter b compared to the letter a, across all length and tokenization settings (Visualized in Figure 5 left). We suspect this discrepancy is due to differences in letter frequency in the natural language, which may affect token-embedding sensitivity in the model. To further investigate how token frequency impacts counting accuracy in LLMs, we conducted additional counting tasks using the most frequent letter in human language,e, and the least frequent letter, z, under the same settings. As shown in Table 2, we observe similar performance differences, with z achieving much higher counting accuracy than e across all proper counting settings (both with CoT and using letter-separated tokenization). We visualize the counting performance between each pair of settings

String-Token Type	len ∈ [80, 100]			
	z	b	r	e
(b)	14.50	13.60	8.90	8.40
(c)	36.00	36.60	28.30	24.30
(d)	61.60	60.20	54.10	51.90
Letter Frequency				
percentage	0.07	1.48	6.02	12.70

Table 4: Counting performance of letters that have very different letter frequency in human language.

for both ab and ez in Figure 5. The accuracy advantage of b over a and z over e is clear and significant, ranging from 3% to 14%. Interestingly, both b (1.5%) and z (0.07%) are lower-frequency tokens compared to a (8.2%) and e (12.7%), indicating that lower-frequency tokens are more sensitive to counting tasks in this context. We hypothesize that lower-frequency tokens contain less embedded information due to fewer occurrences during training, making them easier to identify through the attention mechanism. In contrast, frequent tokens like a may embed more complex information (e.g., letter a can be a positional noun), potentially causing distractions during attention calculations and resulting in lower performance.

Comprehensive Experiments on the Relationship Between Letter Frequency and Counting Performance. To verify these results beyond the letter pairs a, b and e, z, we selected another set of letters with significantly different frequencies in *human languages*, according to Wikipedia: z (0.07%), b (1.48%), r (6.02%), and e (12.70%). We generated counting instances of lengths between 80 and 100—ensuring that each letter appears more than 20 times on average—by uniformly sampling one of the four letters to form each string (e.g., zrrbeez). We then performed counting for each letter in the generated strings. As shown in Table 4, a consistent trend was observed across tokenization types (b)–(d) (excluding (a), as pure BPE was previously shown not to yield meaningful counting results). Specifically, rare tokens consistently outperformed more frequent tokens in natural language, with performance improvements ranging from 6% to 12%. Figure 6 visually compares performance and letter frequency, showing an overlap between frequency and error rate. We suspect that rare letters carry less information in their embeddings, reducing distraction during the attention calculation in the counting process.

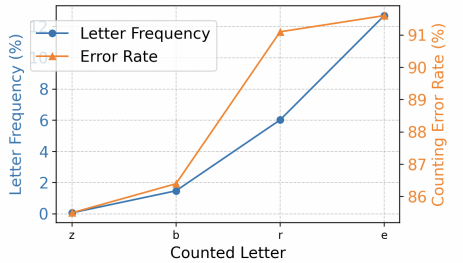

Figure 6: Counting accuracy (Orange) with respect to target letter frequency (Blue) in Human Natural Language.

5.5 Results on Claude and Case Study with GPT-4o

We repeated the experiments with Claude and observed similar trends, with the exception that type (c) yielded the best results, outperforming types (a)-(d), as shown in Table 3. Upon investigation, we suspect this is because type (d) results in significantly longer CoT steps due to the higher number of tokens, leading to long-context reasoning failures in many cases for this model. We also provide case studies using GPT-4o mini for counting tasks, including examples where CoT led to both correct and incorrect answers, as well as cases where the model predicted incorrect answers when CoT was disabled. All case studies are detailed in the Appendix Section D.

To this end, we are confident that our experimental results can be generalized to other LLMs, given that the training methods and tokenization strategies (as demonstrated in Appendix Section C) are nearly identical, leading to counting being performed in a similar manner across such models.

6 Conclusion

This study has demonstrated that tokenization plays a significant role in the reasoning capabilities of large language models (LLMs), particularly in tasks such as counting. The way input data is tokenized influences how models process and interpret information, ultimately affecting performance outcomes. Therefore, careful consideration must be given to the selection of tokenization strategies when working with LLMs, especially in tasks where fine-grained reasoning is essential. Our findings suggest that the exploration and optimization of tokenization methods is not only relevant but crucial for improving model performance. Future research should further investigate the impact of different tokenization techniques, with a focus on refining these methods to enhance LLM reasoning abilities across a broader range of tasks.

Limitations

Our experiments were conducted on GPT-4o and Claude-3.5, and while both models demonstrated strong patterns and consistent evidence showing that certain types of tokenization significantly improve counting performance, we did not extend our testing to other LLMs such as LLaMA or Mistral. This was primarily due to budget and time constraints, as well as preliminary findings that these models exhibited weaker instruction-following abilities compared to GPT and Claude, making the evaluation process more challenging. However, we believe our research remains robust despite these limitations, as mainstream model training and design principles are largely universal, and the patterns observed are likely generalizable to other LLMs.

Additionally, our experiments did not explore extreme context lengths, such as counting instances with more than several hundred tokens. We found that such cases often led to instability due to the accumulation of long CoT steps. We aim to further investigate this aspect as LLMs improve in handling long-context retrieval and generation.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Joshua Ackerman and George Cybenko. 2020. A survey of neural networks and formal languages. *arXiv preprint arXiv:2006.01338*.

Zeyuan Allen-Zhu and Yuanzhi Li. 2023. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609*.

George S Boolos, John P Burgess, and Richard C Jeffrey. 2002. *Computability and logic*. Cambridge university press.

Allan Borodin, Stephen A Cook, Patrick W Dymond, Walter L Ruzzo, and Martin Tompa. 1989. Two applications of inductive counting for complementation problems. *SIAM Journal on computing*, 18(3):559–578.

Yingshan Chang and Yonatan Bisk. 2024. Language models need inductive biases to count inductively. *arXiv preprint arXiv:2405.20131*.

David Chiang and Peter Cholak. 2022. Overcoming a theoretical limitation of self-attention. *arXiv preprint arXiv:2202.12172*.

S Barry Cooper. 2017. *Computability theory*. Chapman and Hall/CRC.

Nicolaas Govert De Bruijn. 1964. Polya's theory of counting. In *Applied combinatorical mathematics*, pages 144–184. Eindhoven University of Technology.

Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, et al. 2022. Neural networks and the chomsky hierarchy. *arXiv preprint arXiv:2207.02098*.

Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Angela Fan, Thibaut Lavril, Edouard Grave, Armand Joulin, and Sainbayar Sukhbaatar. 2020. Addressing some limitations of transformers with feedback memory. *arXiv preprint arXiv:2002.09402*.

Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. 2024. Towards revealing the mystery behind chain of thought: a theoretical perspective. *Advances in Neural Information Processing Systems*, 36.

Patrick C Fischer, Albert R Meyer, and Arnold L Rosenberg. 1968. Counter machines and counter languages. *Mathematical systems theory*, 2(3):265–283.

Karol Gregor, Ivo Danihelka, Andriy Mnih, Charles Blundell, and Daan Wierstra. 2014. Deep autoregressive networks. In *International Conference on Machine Learning*, pages 1242–1250. PMLR.

Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.

Oscar H Ibarra, Jianwen Su, Zhe Dang, Tevfik Bultan, and Richard A Kemmerer. 2002. Counter machines and verification problems. *Theoretical Computer Science*, 289(1):165–189.

Mark Jerrum. 1995. The computational complexity of counting. In *Proceedings of the International Congress of Mathematicians: August 3–11, 1994 Zürich, Switzerland*, pages 1407–1416. Springer.

Zhi Jin, Sheng Xu, Xiang Zhang, Tianze Ling, Nanqing Dong, Wanli Ouyang, Zhiqiang Gao, Cheng Chang, and Siqi Sun. 2024. Contranovo: A contrastive learning approach to enhance de novo peptide sequencing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 144–152.

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *nature*, 521(7553):436–444.

Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. 2024. Chain of thought empowers transformers to solve inherently serial problems. *arXiv preprint arXiv:2402.12875*.

Puyuan Liu, Xiang Zhang, and Lili Mou. 2022. A character-level length-control algorithm for non-autoregressive sentence summarization. *Advances in Neural Information Processing Systems*, 35:29101–29112.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. 2023. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*.

Maithra Raghu, Ben Poole, Jon Kleinberg, Surya Ganguli, and Jascha Sohl-Dickstein. 2017. On the expressive power of deep neural networks. In *international conference on machine learning*, pages 2847–2854. PMLR.

Paul Rodriguez, Janet Wiles, and Jeffrey L Elman. 1999. A recurrent neural network that learns to count. *Connection Science*, 11(1):5–40.

Frank Rosenblatt. 1958. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386.

Clayton Sanford, Daniel Hsu, and Matus Telgarsky. 2024. Transformers, parallel computation, and logarithmic depth. *arXiv preprint arXiv:2402.09268*.

Rico Sennrich. 2015. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*.

Alessandro Sperduti. 1997. On the computational power of recurrent neural networks for structures. *Neural Networks*, 10(3):395–400.

Mirac Suzgun, Sebastian Gehrmann, Yonatan Belinkov, and Stuart M. Shieber. 2019. Lstm networks can perform dynamic counting.

Peter Tiňo, Bill G Horne, C Lee Giles, and Pete C Collingwood. 1998. Finite state machines and recurrent neural networks—automata and dynamical systems approaches. In *Neural networks and pattern recognition*, pages 171–219. Elsevier.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Gail Weiss, Yoav Goldberg, and Eran Yahav. 2018. On the practical computational power of finite precision rnns for language recognition.

Karen Wynn. 1990. Children's understanding of counting. *Cognition*, 36(2):155–193.

Yuwei Yin, Jean Kaddour, Xiang Zhang, Yixin Nie, Zhenguang Liu, Lingpeng Kong, and Qi Liu. 2023. Ttida: Controllable generative data augmentation via text-to-text and text-to-image models. *arXiv preprint arXiv:2304.08821*.

Xiang Zhang, Muhammad Abdul-Mageed, and Laks V. S. Lakshmanan. 2024a. Autoregressive + chain of thought = recurrent: Recurrence's role in language models' computability and a revisit of recurrent transformer.

Xiang Zhang and Dujian Ding. 2024. Supervised chain of thought.

Xiang Zhang, Senyu Li, Bradley Hauer, Ning Shi, and Grzegorz Kondrak. 2023. Don't trust ChatGPT when your question is not in English: A study of multilingual abilities and types of LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7915–7927, Singapore. Association for Computational Linguistics.

Xiang Zhang, Tianze Ling, Zhi Jin, Sheng Xu, Zhiqiang Gao, Boyan Sun, Zijie Qiu, Nanqing Dong, Guangshuai Wang, Guibin Wang, et al. 2024b. π-primenovo: An accurate and efficient non-autoregressive deep learning model for de novo peptide sequencing. *bioRxiv*, pages 2024–05.

Appendix

A Supervised Chain of Thought

Naive Chain of Thought (CoT), which uses a generic "think step by step" prompt for all tasks, poses significant challenges for models in determining the correct steps, especially for complex, multi-step reasoning tasks. To mitigate this confounding factor, we follow previous work and employ Supervised CoT (Zhang and Ding, 2024), as the derivation of steps is not the focus of our research and should not affect performance due to incorrect CoT steps. Below, we define Supervised CoT and explain its application in counting tasks.

A.1 Definition

The search space for solving a task can be viewed as a combination of the prompt space and the answer space. When instructed to perform tasks step by step, language models must devise a step template which is used to determine the actions at each step. This template is crucial for solving tasks, as it specifies what information is processed and how it is computed at each CoT step. However, for a given task, there are numerous ways to perform a "step-by-step" approach, each computing different elements per step. Finding the optimal set of steps is challenging yet essential, as it directly influences the ability to find solutions in the answer space (Zhang and Ding, 2024).

Supervised CoT provides human supervision in determining the step template. Rather than asking the model to develop its own plan for each step, humans identify the "recurrent" procedure in the computation and explicitly instruct the model to follow a specific step template. This approach allows the CoT to bypass the need to search for optimal steps, focusing instead on finding solutions within the answer space under optimal step guidance.

A.2 Supervised CoT and Counting

In inductive counting, which relies on CoT to compute the counter value recurrently(Figure 2), it is crucial that each step of CoT accurately extracts and outputs the counter value in text. This output is necessary for the value to be recurrently processed through "string-vector" conversion. Therefore, rather than simply prompting the model with "determine the number of `a` in the given string" using the generic instruction "think step by step," we specifically instruct the model to print out a counter value at each step. We explicitly define the step template to ensure the model follows the optimal CoT steps, preventing deviations or the use of suboptimal steps.

Experiments. We demonstrate the significant performance gap between Supervised and Unsupervised CoT. Specifically, we observe that supervision not only helps the model accurately extract the counter but also ensures it follows the correct steps (e.g., an incorrect step would be outputting whether the current letter is the target, rather than extracting the counter value). Even when Unsupervised CoT identifies the correct steps (i.e., extracting the counter into text), we still notice more frequent errors during the extraction process compared to

String-token Type	Counting a		Counting b	
	Unsupervised-CoT	Supervised CoT	Unsupervised-CoT	Supervised CoT
(b)	8.40	**10.90**	**20.70**	18.60
(c)	24.00	**28.10**	29.30	**42.30**
(d)	34.90	**56.10**	42.70	**70.80**

Table 5: Counting experiments in the length range of 30-40 comparing Supervised CoT and Unsupervised CoT. The bolded font indicates the better performance in the pairwise comparison between Supervised and Unsupervised CoT.

Supervised CoT, which imposes strict constraints on what to extract at each step. The comparison between Supervised and Unsupervised CoT is presented in Table 5, showing a clear dominance of Supervised CoT, with accuracy gains observed in nearly all cases.

B Replication Experiments Note

We have open-sourced the experimental results for every instance of each experiment, in the provided GitHub link, to facilitate future research and analysis by other researchers. All reported experiment numbers are stable, using the same experimental settings and prompts. Specifically, we observe an average variance in accuracy of less than 1% across runs of the same experiments, indicating that they are fully replicable with the same GPT-4o version used. However, note that updates to the API version may cause potential variations in results, which are beyond our control.

C Tokenization in Different LLMs

Figure 7 illustrates the tokenization of input strings across various LLMs. We investigate both language models and multi-modal models, observing nearly identical tokenization behaviors across all tested models. Specifically, pure strings (e.g., `abbab`) may be tokenized differently due to the varying byte pair encoding (BPE) schemes used by each tokenizer. When a token-level delimiter is introduced, we consistently observe that the delimiter is combined with the adjacent letter, aligning with our previous assumptions. Additionally, some models handle the initial token differently, resulting in the first letter being treated as a standalone token (e.g., in the Grok model) or being combined with a quotation delimiter (e.g., in GPT-4o). In summary, our string design effectively allows us to manipulate modern LLMs to tokenize identical counting instances into different, desired tokens.

D Case Study

Tables 6-9 present cases for each type of token counted using CoT. As shown, Supervised CoT consistently adheres to a strict step-by-step template, accurately extracting the counter at each step. In contrast, Unsupervised CoT often skips crucial steps or deviates from the optimal method of extracting counters. Additionally, Supervised CoT with type (d) tokenization tends to produce much longer reasoning contexts, yet still achieves the best performance due to the combination of optimal tokenization and supervision.

Prompt: Task: Count the number of occurrences of the substring ’a’ in the given string step by step. Do not use bold font in the response. Instructions: Iterate through the string and count each occurrence of the substring. Use a counter to keep track of how many times the substring appears. Output your step-by-step counting process. Conclude with: ’Result: ’ followed by the number. String: ["b", "b", "a", "a", "b". "b", "a", "b", "b", "b", "a", "a", "b", "a", "b", "a", "a", "b", "b", "a", "b"] Response:
Supervised CoT Result (Experiment 4): "1. Initialize a counter to 0.2. Start iterating through the string.3. Check the first element ’b’. It does not match ’a’, so the counter remains 0.4. Move to the second element ’b’. It does not match ’a’, so the counter remains 0.5. Check the third element ’a’. It matches ’a’, so increment the counter to 1.6. Move to the fourth element ’a’. It matches ’a’, so increment the counter to 2.7. Check the fifth element ’b’. It does not match ’a’, so the counter remains 2.8. Move to the sixth element ’b’. It does not match ’a’, so the counter remains 2.9. Check the seventh element ’a’. It matches ’a’, so increment the counter to 3.10. Move to the eighth element ’b’. It does not match ’a’, so the counter remains 3.11. Check the ninth element ’b’. It does not match ’a’, so the counter remains 3.12. Move to the tenth element ’b’. It does not match ’a’, so the counter remains 3.13. Check the eleventh element ’a’. It matches ’a’, so increment the counter to 4.14. Move to the twelfth element ’a’. It matches ’a’, so increment the counter to 5.15. Check the thirteenth element ’b’. It does not match ’a’, so the counter remains 5.16. Move to the fourteenth element ’a’. It matches ’a’, so increment the counter to 6.17. Check the fifteenth element ’b’. It does not match ’a’, so the counter remains 6.18. Move to the sixteenth element ’a’. It matches ’a’, so increment the counter to 7.19. Check the seventeenth element ’a’. It matches ’a’, so increment the counter to 8.20. Move to the eighteenth element ’b’. It does not match ’a’, so the counter remains 8.21. Check the nineteenth element ’b’. It does not match ’a’, so the counter remains 8.22. Move to the twentieth element ’b’. It does not match ’a’, so the counter remains 8.23. Check the twenty-first element ’a’. It matches ’a’, so increment the counter to 9.24. Move to the twenty-second element ’b’. It does not match ’a’, so the counter remains 9. Result: 9
Correct Answer: 9

Table 6: Case Study 1, Correct Prediction with Supervised Chain-of-Thought

Prompt: Count the number appearances of ’a’s in the string below. Directly output ’Result: ’ followed by the counted number. Do not use bold font in the response. String: ["b", "b", "a", "a", "b". "b", "a", "b", "b", "b", "a", "a", "b", "a", "b", "a", "a", "b", "b", "a", "b"] Response:
Result (Experiment 4): Result: 7
Correct Answer: 9

Table 7: Case Study 2, Incorrect Prediction Prompting Banning the usage of Chain-of-Thought.

Prompt: Determine the number of appearances of ’{substring}’s in the string below. Think step by step. Directly output ’Result: ’ followed by the counted number. Do not use bold font in the response. String: b b a a b b b a b b b b b a a a b b b b Response:
Result (Experiment 2): To determine the number of appearances of ’a’s in the given string, we will analyze the string step by step. 1. Let’s first isolate the string and count the occurrences of the letter ’a’. The given string is: "b b a a b b b a b b b b b a a a b b b b" 2. We can count the ’a’s by looking through the string or splitting it into individual elements to count: - The string has the following elements: b, b, a, a, b, b, b, a, b, b, b, b, b, a, a, a, b, b, b, b. 3. Now we count each ’a’: - First occurrence: a (1) - Second occurrence: a (2) - Third occurrence: a (3) - Fourth occurrence: a (4) - Fifth occurrence: a (5) 4. Total occurrences of ’a’: 5 Result: 5
Correct Answer: 6

Table 8: Case Study 3, Incorrect Prediction with Unsupervised Chain-of-Thought

Prompt: Task: Count the number of occurrences of the substring ’a’ in the given string step by step. Do not use bold font in the response. Instructions: Iterate through the string and count each occurrence of the substring. Use a counter to keep track of how many times the substring appears. Output your step-by-step counting process. Conclude with: ’Result: ’ followed by the number. String: bbaabbbabbbbbaaabbbb Response:
Result (Experiment 1): Let’s go through the string "bbaabbbabbbbbaaabbbb" step by step to count the occurrences of the substring ’a’. 1. Initialize a counter to 0. 2. Start reading the string from the first character. - Character 1: ’b’ (no ’a’, counter remains 0) - Character 2: ’b’ (no ’a’, counter remains 0) - Character 3: ’a’ (found ’a’, increment counter to 1) - Character 4: ’a’ (found ’a’, increment counter to 2) - Character 5: ’b’ (no ’a’, counter remains 2) - Character 6: ’b’ (no ’a’, counter remains 2) - Character 7: ’b’ (no ’a’, counter remains 2) - Character 8: ’a’ (found ’a’, increment counter to 3) - Character 9: ’b’ (no ’a’, counter remains 3) - Character 10: ’b’ (no ’a’, counter remains 3) - Character 11: ’b’ (no ’a’, counter remains 3) - Character 12: ’b’ (no ’a’, counter remains 3) - Character 13: ’a’ (found ’a’, increment counter to 4) - Character 14: ’a’ (found ’a’, increment counter to 5) - Character 15: ’b’ (no ’a’, counter remains 5) - Character 16: ’b’ (no ’a’, counter remains 5) - Character 17: ’b’ (no ’a’, counter remains 5) - Character 18: ’b’ (no ’a’, counter remains 5) 3. End of the string reached. Result: 5
Correct Answer: 6

Table 9: Case Study 4, Incorrect Prediction with Supervised Chain-of-Thought

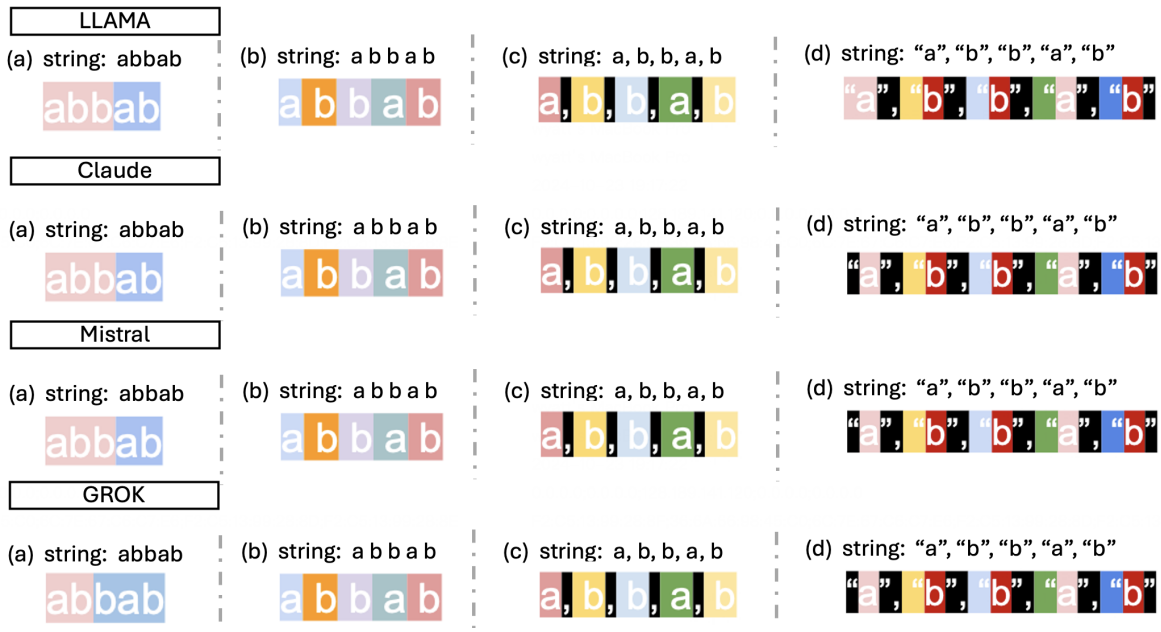

Figure 7: Difference in tokenization on strings when counting instances are presented in different formats, across different LLMs.